

SOFTWARE REQUIREMENTS SPECIFICATION

Internship Progress Dashboard

Field	Details
Document Title	Software Requirements Specification (SRS)
Project Name	Internship Progress Dashboard
Version	1.0
Date	April 2026
Prepared By	Development Team
Reviewed By	Mentor / Project Lead
Status	Draft

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Internship Progress Dashboard — a web-based platform that allows mentors to manage interns and enables students to track their own progress during an internship program.

This document is intended for developers, testers, project managers, and stakeholders who are involved in the design, development, and evaluation of the system.

1.2 Scope

The Internship Progress Dashboard provides the following high-level capabilities:

- Secure role-based user registration and authentication (Mentor and Student)
- Task creation, assignment, and lifecycle management
- Weekly report submission by students with mentor feedback via comments
- Automated in-app notifications triggered by system events
- Role-specific analytics dashboards with visual progress data
- Student assignment management by mentors

The system does NOT cover:

- Email or SMS notifications (in-app only)
- Video/audio communication features
- File/document upload (avatars are URL-based)
- Payment or HR processing

1.3 Definitions, Acronyms & Abbreviations

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
JWT	JSON Web Token — used for stateless authentication
CRUD	Create, Read, Update, Delete — basic data operations
ODM	Object-Document Mapper (Beanie for MongoDB)
Mentor	An instructor/supervisor who manages interns; also acts as the system administrator
Student	An intern registered in the system, assigned to a mentor
FR	Functional Requirement
NFR	Non-Functional Requirement

Term	Definition
UI	User Interface
REST	Representational State Transfer — architectural style for the API

1.4 References

- FastAPI Official Documentation — <https://fastapi.tiangolo.com>
- Beanie ODM Documentation — <https://beanie-odm.dev>
- MongoDB Documentation — <https://www.mongodb.com/docs>
- JWT Standard — RFC 7519
- Project Technical & Testing Document v1.0 (companion document)

1.5 Document Overview

Section 2 provides the overall system description. Section 3 details all functional requirements. Section 4 covers non-functional requirements. Section 5 specifies external interface requirements. Section 6 lists system constraints and assumptions.

2. Overall Description

2.1 Product Perspective

The Internship Progress Dashboard is a new, standalone web application. It is not a replacement or extension of any existing system. It consists of two parts:

- Backend — A RESTful API built with FastAPI (Python) and MongoDB, serving JSON data
- Frontend — A React (Vite) single-page application consuming the backend API

The backend and frontend communicate exclusively over HTTP using JWT-protected REST endpoints. The system can be deployed independently via Docker or as a conventional process.

2.2 Product Functions

The major functions of the system are:

Function Group	Description
User Management	Registration, login, profile update, student assignment to mentor
Task Management	Mentor creates and manages tasks; students update task status
Report Management	Students submit markdown-based weekly reports; mentor adds feedback comments
Notifications	Automatic in-app notifications on task assignment, report submission, and new comments
Analytics	Aggregated progress data per role: task completion rates, weekly activity, per-student breakdown

2.3 User Classes and Characteristics

User Class	Technical Level	Access	Characteristics
Mentor	Moderate	Full system access within own scope	Creates tasks, manages students, reviews reports, monitors analytics. Acts as admin — no separate admin role exists.
Student	Low to Moderate	Restricted to own data	Updates task status, submits weekly reports, reads own analytics. Cannot create tasks or manage other users.
Unauthenticated User	Any	Register / Login only	Can only access /register and /login endpoints. All other endpoints require valid JWT.

2.4 Operating Environment

- Backend: Python 3.11+, FastAPI, Motor/Beanie, MongoDB 6+
- Frontend: Node.js 18+, React 18+, Vite

- Database: MongoDB (local or Atlas cloud)
- Deployment: Docker (backend), or direct uvicorn process
- Browsers: Modern evergreen browsers (Chrome, Firefox, Edge, Safari)
- OS: Windows 10+, Ubuntu 20.04+, macOS 12+

2.5 Design and Implementation Constraints

- Authentication is stateless JWT — no server-side session storage
- All API responses are JSON — no XML or other formats
- MongoDB is the only supported database — no SQL alternative
- The system uses UTC timestamps throughout
- Avatar/profile images are stored as URL strings, not binary blobs
- Password hashing uses bcrypt via passlib

2.6 Assumptions and Dependencies

- A running MongoDB instance is accessible at the configured MONGO_URL
- The frontend runs on port 5173 (Vite default) or port 3000
- Only one mentor can be assigned to a student at a time
- Week numbers in reports are manually entered by the student (not auto-calculated)
- Notifications are not delivered externally — only visible within the dashboard

3. Functional Requirements

Requirements are identified as FR-MOD-NNN where MOD is the module code.

3.1 Authentication Module (FR-AUTH)

ID	Requirement	Priority	Role
FR-AUTH-001	The system shall allow any user to register with a unique email address, full name, password, and role (student or mentor).	High	All
FR-AUTH-002	The system shall reject registration if the email is already associated with an existing account.	High	All
FR-AUTH-003	The system shall store passwords in hashed form (bcrypt) and never in plaintext.	High	All
FR-AUTH-004	The system shall issue a signed JWT access token upon successful login.	High	All
FR-AUTH-005	The JWT token shall embed the user ID and role, and shall expire after a configurable duration (default 1440 minutes).	High	All
FR-AUTH-006	The system shall authenticate all protected endpoints by validating the Bearer token in the Authorization header.	High	All
FR-AUTH-007	The system shall return the current user's profile via GET /api/auth/me when a valid token is provided.	Medium	All
FR-AUTH-008	The system shall return HTTP 401 for any request to a protected endpoint with a missing, invalid, or expired token.	High	All

3.2 Task Management Module (FR-TASK)

ID	Requirement	Priority	Role
FR-TASK-001	The system shall allow mentors to create tasks with a title, optional description, status, priority, assignee (student ID), optional due date, and optional tags.	High	Mentor
FR-TASK-002	On task creation, the system shall automatically create an in-app notification for the assigned student.	High	Mentor
FR-TASK-003	The system shall prevent students from creating tasks; any such attempt shall return HTTP 403.	High	Student
FR-TASK-004	Mentors shall only see tasks they created; students shall only see tasks assigned to them.	High	Both
FR-TASK-005	The system shall support filtering tasks by status (todo, in_progress, completed) via a query parameter.	Medium	Both
FR-TASK-006	The system shall support keyword search across task title, description, and tags via a search query parameter.	Medium	Both

ID	Requirement	Priority	Role
FR-TASK-007	Mentors shall be able to update all fields of tasks they created.	High	Mentor
FR-TASK-008	Students shall only be able to update the status field of tasks assigned to them; all other field updates shall be silently ignored.	High	Student
FR-TASK-009	Only the mentor who created a task shall be able to delete it.	High	Mentor
FR-TASK-010	Attempting to access a non-existent task ID shall return HTTP 404.	Medium	Both
FR-TASK-011	Task status shall support three values: todo, in_progress, completed.	High	Both
FR-TASK-012	Task priority shall support three values: low, medium, high.	High	Mentor

3.3 Report Management Module (FR-RPT)

ID	Requirement	Priority	Role
FR-RPT-001	The system shall allow students to submit weekly reports containing a title, Markdown content, and a week number.	High	Student
FR-RPT-002	On report submission, the system shall create an in-app notification for the student's assigned mentor (if mentor_id is set).	High	Student
FR-RPT-003	Students shall be able to update or delete only their own reports.	High	Student
FR-RPT-004	Mentors shall only see reports submitted by their assigned students.	High	Mentor
FR-RPT-005	Both mentors and students shall be able to add comments to a report.	High	Both
FR-RPT-006	When a mentor adds a comment on a student's report, the system shall create a notification for that student.	High	Mentor
FR-RPT-007	A user shall only be able to delete their own comments.	High	Both
FR-RPT-008	Comments shall be stored embedded within the report document (not as a separate collection).	Medium	System
FR-RPT-009	Mentors shall not be able to submit reports; any such attempt shall return HTTP 403.	High	Mentor
FR-RPT-010	Attempting to access a non-existent report ID shall return HTTP 404.	Medium	Both

3.4 Analytics Module (FR-ANA)

ID	Requirement	Priority	Role
FR-ANA-001	The system shall return different analytics payloads depending on the authenticated user's role.	High	Both
FR-ANA-002	Mentor analytics shall include: total assigned students, total tasks created, task status breakdown (todo/in_progress/completed), overall completion rate, weekly activity for the last 7 weeks, and per-student progress.	High	Mentor
FR-ANA-003	Student analytics shall include: total assigned tasks, total submitted reports, task status breakdown, completion rate, weekly activity for the last 7 weeks, priority breakdown, and total feedback received.	High	Student
FR-ANA-004	Weekly activity shall cover exactly 7 consecutive weeks ending at the current date.	Medium	Both
FR-ANA-005	Completion rate shall be calculated as (completed tasks / total tasks * 100) and rounded to one decimal place. If total tasks is zero, completion rate shall be 0.0.	High	Both
FR-ANA-006	Per-student progress for mentors shall show each student's name, total tasks, completed count, and completion rate.	High	Mentor

3.5 User Management Module (FR-USR)

ID	Requirement	Priority	Role
FR-USR-001	Mentors shall be able to retrieve a list of students currently assigned to them.	High	Mentor
FR-USR-002	Mentors shall be able to retrieve a list of all registered students (for assignment purposes).	High	Mentor
FR-USR-003	Mentors shall be able to toggle the assignment of a student: if unassigned, assign to self; if already assigned, unassign (set mentor_id to None).	High	Mentor
FR-USR-004	Any authenticated user shall be able to update their own profile: name, bio, skills list, and avatar URL.	Medium	Both
FR-USR-005	Profile updates shall be partial — only supplied fields shall be updated.	Medium	Both
FR-USR-006	Students shall not be able to access student listing or assignment endpoints; such requests shall return HTTP 403.	High	Student

3.6 Notification Module (FR-NOTIF)

ID	Requirement	Priority	Role
FR-NOTIF-001	The system shall automatically create notifications for the following events: task assigned to student, student submits a report, mentor comments on a student's report.	High	System

ID	Requirement	Priority	Role
FR-NOTIF-002	Each notification shall have: user_id (recipient), title, message, type, read (boolean, default false), and created_at.	High	System
FR-NOTIF-003	The system shall return the latest 30 notifications for the authenticated user, sorted by creation date descending.	High	Both
FR-NOTIF-004	The system shall provide an endpoint to return the count of unread notifications for the current user.	Medium	Both
FR-NOTIF-005	Users shall be able to mark a single notification as read.	Medium	Both
FR-NOTIF-006	Users shall be able to mark all their notifications as read in one request.	Medium	Both
FR-NOTIF-007	A user shall not be able to read or modify notifications belonging to other users.	High	Both

4. Non-Functional Requirements

4.1 Performance

ID	Requirement
NFR-PERF-001	API responses for standard CRUD operations shall complete within 500ms under normal load (up to 50 concurrent users).
NFR-PERF-002	Analytics endpoints may take up to 2 seconds for accounts with 100+ tasks.
NFR-PERF-003	The notification listing endpoint shall return within 300ms for up to 30 records.

4.2 Security

ID	Requirement
NFR-SEC-001	All passwords shall be hashed with bcrypt before storage; plaintext passwords shall never be logged or stored.
NFR-SEC-002	JWT tokens shall be signed using a secret key stored in environment variables, never hardcoded.
NFR-SEC-003	All protected endpoints shall validate the JWT token on every request.
NFR-SEC-004	The CORS policy shall be restricted to known frontend origins in production environments.
NFR-SEC-005	Sensitive fields such as password hash shall never be returned in API responses.
NFR-SEC-006	Rate limiting shall be applied to authentication endpoints (/register, /login) to prevent brute-force attacks.

4.3 Reliability & Availability

ID	Requirement
NFR-REL-001	The system shall maintain uptime of at least 99% during business hours (09:00–18:00 IST, Monday–Friday).
NFR-REL-002	Database connection failures shall be handled gracefully with appropriate HTTP 500 responses, not application crashes.
NFR-REL-003	All database operations shall be async (using Motor) to avoid blocking the event loop.

4.4 Usability

ID	Requirement
NFR-USA-001	The API shall expose interactive documentation at /api/docs (Swagger UI) for developer reference.

ID	Requirement
NFR-USA-002	All API error responses shall include a human-readable 'detail' field in the JSON body.
NFR-USA-003	Date/time fields in API responses shall be formatted as ISO 8601 strings.

4.5 Maintainability

ID	Requirement
NFR-MNT-001	The backend codebase shall be organized into separate modules: models, routes, schemas, utils.
NFR-MNT-002	Environment-specific configurations (DB URL, secret key, token expiry) shall be stored in .env files and not hardcoded.
NFR-MNT-003	The backend shall include a seed script to populate the database with test data for development purposes.

4.6 Portability

ID	Requirement
NFR-PRT-001	The backend shall include a Dockerfile for containerized deployment.
NFR-PRT-002	The system shall run on any OS supporting Python 3.11+ and Node.js 18+.
NFR-PRT-003	All dependencies shall be declared in requirements.txt (backend) and package.json (frontend).

5. External Interface Requirements

5.1 User Interfaces

- The frontend shall be a React single-page application (SPA) served on `http://localhost:5173` (development) or a configured production URL.
- The application shall provide separate views/dashboards for Mentor and Student roles.
- The application shall be responsive and usable on both desktop and mobile browsers.
- All date/time values displayed to users shall be converted from UTC to the user's local timezone by the frontend.

5.2 Hardware Interfaces

No direct hardware interfaces. The system requires standard network connectivity between the client browser, the API server, and the MongoDB instance.

5.3 Software Interfaces

Interface	Type	Purpose
MongoDB / Motor	Database Driver	Async document storage and retrieval
Beanie ODM	ORM Layer	Schema definition and document validation
FastAPI	Web Framework	HTTP routing, request/response handling
Pydantic v2	Data Validation	Input schema validation and serialization
python-jose / passlib	Security Libraries	JWT creation/verification and password hashing
React (Vite)	Frontend Framework	SPA rendering and API consumption

5.4 Communication Interfaces

- All client-server communication shall use HTTPS in production (HTTP is acceptable in development).
- The API shall use JSON as the data interchange format for all request and response bodies.
- Authentication shall use the HTTP Authorization header with the Bearer scheme.
- The system does not use WebSockets — all communication is request/response based.

6. System Constraints and Limitations

#	Constraint / Limitation	Impact
1	No pagination on list endpoints (/tasks/, /reports/)	Performance degrades with large datasets; all records loaded in memory
2	No email or push notifications — in-app only	Users must be logged in to receive notifications
3	Single mentor per student (mentor_id is a scalar, not a list)	A student cannot be supervised by multiple mentors simultaneously
4	No role upgrade path — a user's role is fixed at registration	A student cannot later become a mentor without re-registering
5	No token refresh mechanism — expired JWT requires re-login	Long sessions will be interrupted; user experience may be affected
6	Avatar stored as URL string only	No file upload support; users must host images externally
7	Week number in reports is manually entered	No validation that the week number is unique or sequential per student
8	analytics endpoint does not support date range filtering	Always returns last 7 weeks; historical data beyond this window is inaccessible in aggregated form

7. Appendix

7.1 Requirements Traceability Matrix

Requirement ID	Module	API Endpoint(s)	Priority
FR-AUTH-001 to 004	Authentication	POST /api/auth/register	High
FR-AUTH-004 to 006	Authentication	POST /api/auth/login	High
FR-AUTH-007 to 008	Authentication	GET /api/auth/me	High
FR-TASK-001 to 002	Task	POST /api/tasks/	High
FR-TASK-004 to 006	Task	GET /api/tasks/	High
FR-TASK-007 to 010	Task	PUT /api/tasks/{id}	High
FR-TASK-009	Task	DELETE /api/tasks/{id}	High
FR-RPT-001 to 003	Report	POST /api/reports/	High
FR-RPT-004	Report	GET /api/reports/	High
FR-RPT-005 to 007	Report	POST /api/reports/{id}/comments	High
FR-ANA-001 to 006	Analytics	GET /api/analytics/	High
FR-USR-001 to 006	User	GET,PUT /api/users/*	High

Requirement ID	Module	API Endpoint(s)	Priority
FR-NOTIF-001 to 007	Notification	GET,PUT /api/notifications/*	High

7.2 Revision History

Version	Date	Author	Changes
1.0	April 2026	Development Team	Initial SRS — all modules documented

--- End of Document ---